

A - Event registration

```
button.addEventListener("click", greet());
```

`greet()` is *called immediately* (note the parentheses) when this line runs, so "Hello" logs once, right away, during setup - not on click. `greet()` returns undefined, so `addEventListener` actually registers undefined as the handler. A later click does nothing, because there's no function attached.

Fix: `button.addEventListener("click", greet);` - pass the function reference, don't call it.

B - Independent closures

```
console.log(a(), a(), b(), a());
```

Output: 1 2 1 3

Each call to `makeCounter()` creates its own `n` in its own closure. `a` and `b` close over *separate* variables, so `b()`'s first call starts back at 1 regardless of how many times `a()` has already run. The third value (`b()` → 1) differs from the second (`a()` → 2) because they're tracking completely independent state, not a shared counter.

C - Event loop order

A
B
C
D

`console.log("A")` and `console.log("B")` are synchronous - they run immediately, in order, before anything else gets a chance.

`Promise.resolve().then(...)` schedules a **microtask**. Microtasks run as soon as the current synchronous code finishes, before any macrotask.

`setTimeout(..., 0)` schedules a **macrotask** (a timer task). Even with a 0ms delay, it's queued to run only after the call stack is empty *and* all pending microtasks have drained.

So order is: sync (A, B) → microtask (C) → macrotask (D). A zero-delay timeout doesn't mean "run immediately" - it means "run as soon as possible *after* the current script and all microtasks are done."

D - Promise chaining

```
Promise.resolve(3)
  .then(n => { n * 2; })
  .then(value => console.log(value));
```

Logs undefined. The first `.then` callback uses a block body (`{ n * 2; }`) with no return, so it implicitly returns undefined, which is what the second `.then` receives.

Fix:

```
Promise.resolve(3)
  .then(n => n * 2)      // or: .then(n => { return n * 2; })
  .then(value => console.log(value)); // logs 6
```

return is what carries a value forward through a `.then` chain - without it, the resolved value silently becomes undefined for the next link.

E - Rejection and cleanup

```
test();
console.log("outside");
```

Output order: outside, then Offline, then cleanup.

success never logs - the `await Promise.reject(...)` throws, so control jumps straight to `catch`, which logs the error message, then finally runs and logs `cleanup`.

"outside" logs *first* because `test()` is an async function: as soon as it hits `await`, it suspends and returns control to the caller immediately (its remaining code becomes a queued continuation). The synchronous `console.log("outside")` right after the call runs before that continuation is processed.

F - Class inheritance

```
class TimedResource extends Resource {
  constructor(title, minutes) {
    this.minutes = minutes; // using `this` before super() causes an error
    super(title);
  }
}
```

This throws a `ReferenceError: Must call super constructor in this before accessing 'this' or returning from derived constructor`. In a derived class, `this` isn't initialized until `super()` runs (that's what actually creates the instance via the parent constructor), so you can't touch `this` beforehand.

Fix - call `super()` first, then set your own fields:

```
constructor(title, minutes) {
  super(title);
  this.minutes = minutes;
}
```

This matters because `super(title)` is what sets up `this.title` (and the object itself); everything a subclass adds must come after that handoff completes.

Test record and Submission

Test	Action	Expected Result
T1	Open or reload the page	Ready; attempts 0; response clicks 0; list and trace empty
T2	Submit with failure unchecked	Loading appears immediately; attempts 1; load button disabled; list cleared.
T3	Click Test page response twice while loading	Response clicks reaches 2 before loading finishes; request count stays 1
T4	Wait for the successful request	Three labels appear in the specified order; Loaded 3 resources; button enabled; trace A B C D E F with E success.
T5	Submit another successful request	Attempts 2; three rows after completion, not six; response clicks stays 2.
T6	Check failure and submit	Attempts 3; list cleared; failure message appears; button enabled; trace A B C D E F with E failure.
T7	Uncheck failure and retry	Attempts 4; success recovers with three rows; no old error remains.
T8	Start unchecked then check failure during loading	Attempts 5; this request still succeeds because the mode was captured at submission.
T9	Uncheck failure and rapidly double click Load	Only one new request is accepted; attempts 6; one final set of three rows
10	Reload the page	Return to the T1 state. Counters and results are stored only in memory.

Screenshots

T3

Class Resource Loader

☐ Simulate failure for this request Load resources

Attempts: 2

Loading...

Test page response

Response clicks: 2

Execution trace

A: handler starts
B: before await
C: promise microtask
D: timer task

T4

Class Resource Loader

☐ Simulate failure for this request Load resources

Attempts: 1

Loaded 3 resources.

- JavaScript Events | 15 min
- Closures Practice | 20 min
- Promises and Await | 25 min

Test page response

Response clicks: 1

Execution trace

```
A: handler starts
B: before await
C: promise microtask
D: timer task
E: success
F: cleanup
```

T6

Class Resource Loader

☒ Simulate failure for this request Load resources

Attempts: 2

Resources could not be loaded.

Test page response

Response clicks: 1

Execution trace

```
A: handler starts
B: before await
C: promise microtask
D: timer task
E: failure
F: cleanup
```

T7

Class Resource Loader

☐ Simulate failure for this request Load resources

Attempts: 4

Loaded 3 resources.

- JavaScript Events | 15 min
- Closures Practice | 20 min
- Promises and Await | 25 min

Test page response

Response clicks: 2

Execution trace

```
A: handler starts
B: before await
C: promise microtask
D: timer task
E: success
F: cleanup
```

Individual Exit Responses

1. Which variable is retained by the counter closure, and why does its value survive between accepted requests?

The count variable declared inside makeCounter() is retained. Each returned function keeps a live reference to its own count in the closure's scope, so the variable isn't garbage-collected between calls - it persists in memory for as long as the returned function (nextAttempt or nextPing) exists.

2. Why do B and C appear before D in the execution trace?

B is logged synchronously, so it runs before the event loop even considers queued tasks. C is a microtask (from Promise.resolve().then), and microtasks always drain completely before the next macrotask runs. D is a macrotask (setTimeout), so it has to wait for both the synchronous code and the microtask queue to finish first.

3. What exactly pauses at await, and what can still happen while the request is pending?

await pauses execution of the async submit handler function - not the whole page. The rest of the JavaScript engine, including the event loop, keeps running, so clicks on the "Test page

response" button (and its own closure counter) still fire and update the pings display while the 1200ms mock request is in flight.

4. How do `catch` and `finally` differ when a request fails?

`catch` only runs when the awaited promise rejects - it's where the error is handled, e.g. showing `error.message` and logging `E: failure`. `finally` runs *regardless* of success or failure, so it's used for cleanup that must always happen: resetting `busy` and re-enabling the load button.

5. What do `extends`, `super(title)`, and `super.describe()` each contribute to `TimedResource`?

`extends Resource` sets up the prototype chain so `TimedResource` inherits `Resource`'s methods. `super(title)` calls the parent constructor to initialize `this.title` before `TimedResource` can set its own `this.minutes`. `super.describe()` calls the parent's original `describe()` (returning just the title) so the overridden method can build on it rather than duplicating that logic, producing `"Title | X min"`.